

Hochschule Landshut
Fakultät Informatik – Studiengang Informatik

Praxisorientiertes Studienprojekt
Individualdokumentation - ILSE

Name: Preitenwieser Lukas
Datum: 28.09.2026

Inhaltsverzeichnis

Thema	Seite
1. Aufgaben.....	3
2. Verantwortungsbereiche.....	4
3. Projektverlauf.....	5-11
3.1 Landing.....	5
3.2 Header und Footer.....	6
3.3 Impressum.....	7
3.4 Login und Signup.....	8
3.5 Room-Seiten.....	9
3.6 Profil.....	10
3.7 Projekt-Abschluss.....	11
3. Fazit.....	12

1. Aufgaben

Der mir zugeordnete Aufgabenbereich ist der Fachlich/Pädagogische.

Daher habe ich die, in dem Projekt enthaltenen Fragen an die Benutzer der Web-Applikation miterstellt, auf Korrektheit und Angemessenheit für unsere Zielgruppe, also Schüler ab der siebten Klasse, überprüft und bei Abweichungen angepasst.

Orientiert habe ich mich dabei an dem Informatik Buch der siebten Klasse.

Außerdem habe ich das Layout, den Aufbau und textuellen Inhalt des Frontends erstellt und die zu den Aufgaben zugehörigen Hilfstexte verfasst.

2. Verantwortungsbereiche

Die ausgeführten Aufgaben waren das Erstellen und Verwalten des Web-Frontends der Applikation.

Speziell das Erstellen des Headers inklusive Navigation, Footer, Impressum, Landing-Page, Rooms-Page, Room-View-Page, Profil und der Invalid-Page Seiten.

Ebenfalls habe ich die Frontend-Library Funktionen erstellt, welche über das von dem backend zur Verfügung gestellte Application Programming Interface, kurz API, mit dem backend kommunizieren und dadurch das Frontend unterstützen.

In der Frontend-Library sind folgende Funktionen enthalten:

- api-handler, welcher Daten von der API mittels GET-request empfängt und Daten durch POST-request zu der API sendet
- data-handler, der die Daten des api handler weiterverarbeitet und der applikation zur Verfügung stellt
- navigation-handler, welcher die Navigationen innerhalb der Frontend-Applikation verwaltet
- input-handler, welcher strings normalisiert und escaped
- sessionStorage-handler der die, von der API empfangenen Themen in dem session Storage des Browsers cached.

Außerdem in der Library enthalten sind Zustand enthaltende Strukturen, wie die AnswerState, welche den Status der Fragen in den Räumen enthält und der ManagementType, welcher verfügbare Optionen bei der Benutzerverwaltung enthält.

Desweiteren ist eine globale styles.scss in der Library enthalten, welche für das Styling der App verwendet wird. In Dieser sind die Schriftart, Schriftgröße und Farben für den light-mode und dark-mode festgelegt.

3. Projektverlauf

3.1 Ergebnis: Landing-Page

Zu Beginn des Projekts habe ich mit vite ein React-Projekt erstellt, Typescript installiert und die Projekt-Struktur aufgestellt.

Eines der ersten Schritte war das Vorbereiten der Links mit dem nativen react BrowserRouter und das Erstellen der ersten Komponenten, wie der Landing-Page. Die Landing-Page habe ich mit Typescript und HTML erstellt und mit Sass stilisiert.

Die Landing-Page wird in zwei Komponenten gegliedert, welche ich als Typescript-Komponenten erstellt und mit Sass stilisiert habe.

Die erste Komponente ist die InfoPage. Diese begrüßt den Benutzer mit dem Namen der Applikation und einem kurzen Text, welcher den Benutzer informiert, wozu die Applikation gedacht ist.

In der zweiten Komponente, der CardsPage, gibt es für jedes Themengebiet eine anklickbare Karte, welche durch die Objekte in dem von dem Backend empfangenen JSON-Objekt generiert werden und den Namen des Themas, eine kurze Erklärung Dieses und ein passendes Icon enthält. Diese Karten habe ich wiederum als einzelne Komponenten in Typescript implementiert um die Logik der Applikation durch Modularisierung einfach zu halten und dienen als Navigationselement, indem Sie den Benutzer auf die jeweilige Content-Seiten weiterleiten.

Herausforderungen

Da ich mit den ersten Komponenten begonnen hatte, bevor ich den Router in der App.tsx erstellt hatte, musste ich in diesem Schritt einiges an dem Frontend umbauen, damit der Router korrekt funktioniert und die Projekt-Stuktur erhalten bleibt.

3.2 Ergebnis: Header und Footer

In diesem Schritt habe ich die Bibliotheks Funktionen, wie den apiHandler, die Sass und css styles library, den dataHandler, interfaceHandler und die React-Komponenten für den Header und Footer der Applikation erstellt und Diese in Typescript, HTML und Sass implementiert.

Der Header besteht einerseits aus der Header-Bar, welche auf jeder Unterseite der Applikation zu sehen ist und andererseits aus der Navigation, welche durch das Hamburger-Menü aufrufbar ist. Dieses habe ich mit Sass implementiert und animiert und mit dem React useState hook die Funktion des Navigation toggle Elements implementiert.

Der Footer ist ebenfalls eine kleinere Version der Navigation, indem Dieser nur auf die Content-Seiten verweist während man durch den Header auf jede Seite der Applikation zugreifen kann.

Zuerst habe ich die Komponenten mit Typescript und HTML erstellt und Diese mit scss für zwei Display-Modi, mobil als Standard und PC für größere Displays stilisiert. Den Schwellenwert habe ich in der scss library styles.scss festgelegt.

Danach habe ich die, für die Navigation benötigten JSON interfaces, die API-kommunikations Funktionen und zugehörige JSON Testdaten in der apiHandler Datei erstellt, um die Navigation implementieren und testen zu können.

Für den Farbwechsel der Applikation habe ich farben für den light-mode und dark-mode und highlight Farben in der Sass Bibliothek festgelegt und den, zu dem Color-Change zugehörigen Code, in dem Header vorbereitet und implementiert.

Außerdem habe ich die Sass Bibliothek der Applikation erstellt, in der Variablen für die Schriftart festgelegt werden, das globales style Element index.scss geschrieben und die zugehörige Sass Bibliothek in der Application-Root eingebunden um den Farbenwechsel global auf jeder Unterseite umsetzen zu können.

Herausforderungen

Dadurch, dass die Planung zu diesem Zeitpunkt noch nicht beendet war, war es wichtig, das Team über die benötigten JSON Definitionen zu informieren, um sicherzustellen, dass die API des Backends mit dem Frontend kompatibel ist und um zu vermeiden, fertiggestellte Teile des Frontends später neu implementieren zu müssen.

3.3 Ergebnis: Impressum

Nachdem die erste Seite inklusive Header und Footer fertiggestellt war, habe ich die Impressum und InvalidPage Seiten erstellt, den Typescript und HTML Code geschrieben und die Seiten mit Sass stilisiert.

Das Impressum besteht aus einem Text Componenten, welcher unser Team und unsere Absichten mit dem Entwickeln der Applikation beschreibt und einem Textfeld für eingeloggte Benutzer, in dem Diese Feedback hinterlassen können.

Da es auf der Impressums-Seite ein Textfeld für die Eingabe von Kommentaren gibt, habe ich die stringsHandler Bibliothek geschrieben, welche die Benutzereingabe filtert und normalisiert. Diese Funktion wird später ebenfalls dazu benutzt, um mit der API zu kommunizieren. Mit der Funktion aus dieser Bibliothek habe ich die Eingabe gefiltert und diese an die neu geschriebene apiHandler Funktion für das Versenden von Feedback weitergeleitet. Dazu habe ich die zugehörige JSON Definition in der interfaceHandler Datei erstellt.

Das Prüfen, ob ein Benutzer eingeloggt ist wird durch die Methode isLoggedIn in der globalVars Datei festgestellt, welche ich für diesen Zweck geschrieben habe. Dazu habe ich die globalVars Datei erstellt und um Methoden und Variablen ergänzt, welche den Login-Token und Login-Status verwalten und die Url und Sub-Urls des Backends enthalten.

Die InvalidPage-Seite besteht aus einem Text-Element, welches den Benutzer informiert, dass die angeforderte Seite nicht verfügbar ist und Diesen bei klicken auf den Text auf die zuletzt besuchte Unterseite der Applikation weiterleitet.

3.4 Ergebnis: Login und Signup

Anschließend ging es mit der Implementierung der Login und Signup Seiten weiter. Dazu habe ich die Ordnerstruktur erstellt, die Seiten Struktur und den Seiteninhalt in der React Komponenten Datei mit Typescript und HTML festgelegt und die Login-Seite und Signup-Seite mit Sass stilisiert.

Nach dem Stylen habe ich den Wechsel zwischen der Login Funktion und der Signup Funktion mit Typescript implementiert und den apiHandler um die, dem Einloggen und Registrieren zugehörigen Funktionen ergänzt.

Mit Hilfe des stringsHandler habe ich den login und signup Mechanismus inklusive textuelle Benutzer-Benachrichtigungen implementiert und den apiHandler um die Funktion für das Einloggen und Registrieren erweitert. Unterstützt wird diese Funktion durch die Methoden isLoggedIn, getLoginToken und setLoginState in der globalVars Datei.

Anschließend habe ich das Seitenverhalten geändert, um den Benutzer nach einer erfolgreichen Registrierung automatisch auf die Login Seite weiterzuleiten um die Benutzererfahrung zu Verbessern.

In diesem Fall wird eine Nachricht an den Benutzer ausgegeben. Außerdem in diesem Schritt habe ich die Funktion implementiert, welche den Benutzer nach dem Abmelden auf die Landing-Page weiterleitet. Beide dieser Funktionen werden durch den zuvor implementierten navigationHandler unterstützt.

Herausforderungen

Der Mechanismus zum Einloggen und Registrieren war zu dieser Zeit noch nicht Backend-seitig implementiert, daher musste ich die JSON Definitionen festlegen, mit dem Team kommunizieren und dafür sorgen, dass das Frontend einen Mechanismus hat, den login-Status zu verfolgen. Dazu habe ich die globalVars Datei und die zugehörigen Methoden und Variablen erstellt.

3.5 Ergebnis: Room-Seiten

Nun habe ich mit der Entwicklung der eigentlichen Content-Seiten der Applikation begonnen. Dazu habe ich für jede logische Komponente dieser Seite eine Ordnerstruktur erstellt, die Komponenten mit Typescript und HTML erstellt und Diese mit Sass stilisiert.

Außerdem in diesem Schritt habe ich die JSON-Definitionen für das Empfangen des Seiteninhalts durch das Backend und das Senden des Bearbeitungsstatus der Aufgaben an das Backend in der interfaceHandler Datei festgelegt und passende JSON-Objekte in der apiHandler Datei erstellt.

Ein Room besteht hierbei aus dem Room-Namen, dem Room-Icon und mehreren Topics, die aus dem, von dem Backend empfangenen JSON-Objekt ausgelesen werden um darauffolgend in die React-Komponente RoomTopic übersetzt zu werden. Ein Topic besteht aus der TopicDescription und aus der TopicExercise.

Um die Aufgabe zu erstellen wird, je nach Definition der JSON-Daten, der Exercise-Type ausgelesen und in der TopicExercise Klasse die passende Exercise anhand des JSON-Objekts generiert.

Um die Seite einfach zu halten habe ich in diesem Schritt nur Code für die Multiple-Choice und Linking Exercises geschrieben und darauf geachtet, dass die Applikation für weitere Exercise-Arten erweiterbar bleibt.

Die Multiple-Choice Exercise wird modular aus dem JSON Objekt aufgebaut und erstellt für jedes exerciseAnswer Objekt der JSON einen Typescript-Komponenten in Form des SingleChoiceElement.

Die Überprüfung der Korrektheit der Aufgaben habe ich durch die Benutzung des React useEffect Hooks implementiert. So wird bei dem Click auf den "Abgeben" Text innerhalb des Topic-Elements ein Ereignis erzeugt, welches bis zu dem Kindelement

MultipleChoiceExercise weitergeben wird. Sobald Dieses das Ereignis empfängt überprüft es die Korrektheit der Aufgabe und gibt diesen Status durch ein weiteres Ereignis an die Topic-Klasse weiter, welche den Status in Form von Text und einer Progress-Bar an den Benutzer ausgibt. Die Fortschrittsanzeige das Bild des Raumes, sowie das Dreieck, das den Öffnungsstatus eines Topics anzeigt und die farbliche Hinterlegung bearbeiteter Topics habe ich durch Sass implementiert und animiert.

3.6 Ergebnis: Profil

In diesem Schritt habe ich die Ordnerstruktur für die Profil Seite erstellt, den Typescript und HTML code geschrieben und Diese mit Sass stilisiert. Begonnen habe ich damit, die JSON-Definition für die Profil Seite in der interfaceHandler Datei festzulegen und ein JSON Testobjekt in der apiHandler Datei zu erstellen. Außerdem in dem apiHandler habe ich Methoden hinzugefügt, mit denen der Benutzer den eigenen Account und, sollte dieser die Rolle des Lehrers haben, die Accounts dessen Schüler verwalten kann.

Danach habe ich die Profil Seite in ihre logischen Komponenten aufgeteilt und für jede der Komponenten eine Ordnerstruktur angelegt. Zuerst habe ich das HTML-Layout für jede Komponente in der React-Komponenten Datei festgelegt und jedes Layout mit scss stilisiert. Anschließend habe ich die Funktionen der RoomsStatusPage erstellt, welche den Fortschritt des eigenen Benutzers und, sollte der Benutzer ein Lehrer sein, den Fortschritt der Schüler anzeigt. Anschließend habe ich das StudentDashboard erstellt, welches anhand des JSON-Objektes Karten für jeden Schüler erstellt und diese mit Typescript implementiert.

Diese Karten sind Auf- und Zuklappbar um die Übersichtlichkeit der Seite zu gewährleisten und enthalten neben der zuvor erwähnten RoomsStatusPage den Namen des Schülers und Optionen zum Zurücksetzen des Passworts und Löschen des Accounts. Bei den zuletzt genannten Funktionen wird der Benutzer durch eine Alert Box aufgefordert die Wahl zu bestätigen, um ungewollte Account-Administration zu verhindern. Diese Elemente habe ich ebenfalls in diesem Schritt mit Typescript, HTML und Sass erstellt.

Ebenfalls eingesetzt werden die Alert Boxen in der als nächstes erstellten AccountControl Komponente. Durch Diese kann man das eigene Profilbild ändern, den eigenen Account löschen und das eigene Passwort ändern. Um das Profilbild und das Passwort ändern zu können habe ich dem Projekt zwei weitere Alert-Boxen, die UserIconAlertBox und PasswordAlertBox, hinzugefügt.

Danach habe ich ein Kontrollelement hinzugefügt, durch den ein Lehrer eine von Diesem bestimmte Anzahl an Schüler hinzufügen und die erstellten Accounts durch die Auflistung von Benutzernamen und temporäres Passwort an seine Schüler weitergeben kann. Anschließend habe ich mit typescript den Mechanismus implementiert, anhand dem entschieden wird welche Elemente des Profils durch das JSON-Objekt generiert und angezeigt werden.

3.7 Ergebnis: Projekt-Abschluss

Zuletzt habe ich in dem apiHandler die Methoden geschrieben, welche über die JSON-API mit dem Backend kommunizieren und den API-Handler auf die Benutzung des Backends vorbereitet.

Außerdem habe ich mehrere Fehler im Design behoben und den Sass code der Applikation vor allem im mobilen Design-Modus verbessert.

Desweiteren habe ich der Applikation Bilder auf der Landing-Seite, Icons in den Räumen und Profilbilder hinzugefügt, die Schriftgrößen verbessert, jeder Unterseite der Applikation einen Titel im Browser-Header zugewiesen und die Funktionen der Webseite geprüft.

Herausforderungen:

Dadurch, dass das Backend bis zu dem Projektende nicht testbereit war konnte ich das Frontend nicht mit den Backend verbinden und daher die API-Funktionen nicht vervollständigen und testen.

4. Fazit

Insgesamt bin ich mit dem Ergebnis des Projekts und meinem Aufgabenbereich, der Frontendentwicklung zufrieden, allerdings finde ich es schade, dass die Webseite nicht wie anfangs geplant mit dem Backend verbunden und damit fertiggestellt werden konnte.

Von Vorteil ist jedoch, dass wir als Team durch das Studienprojekt einiges an Erfahrung sammeln konnten, sowohl die technischen Themen der Programmierung und Projektmanagement als auch der Teamarbeit betreffend.

Bei der Teamarbeit hätte ich mir eine nähere Zusammenarbeit zwischen Frontend und Backend gewünscht. Dadurch hätten wir den derzeitigen Zustand vermeiden können, dass sich Frontend und Backend unabhängig entwickeln und eine Zusammenführung dieser kompliziert wird.

Abschließend kann ich sagen, dass mir das Projekt Freude bereitet hat und das Verwalten unseres Frontend-Teams durch eine klare Aufgabenteilung gut funktioniert hat.